

Kibernetinio Saugumo Pilnas Žinynas – II Dalis

Docker ir Konteinerių Saugumo Vadovas

```
## 1. Saugus Dockerfile ```dockerfile FROM python:3.12-slim RUN groupadd -g 10001 appgroup && \ useradd -u 10001 -g appgroup -s /sbin/nologin -M appuser WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt COPY --chown=appuser:appgroup . USER appuser EXPOSE 8080 CMD ["python", "bot.py"] ```
```

```
##2. Konteinerio paleidimo apsauga (Runtime Hardening) ```bash docker run -d \ --name saugi_aplikacija \ --restart unless-stopped \ --security-opt=no-new-privileges:true \ --cap-drop=ALL \ --cap-add=NET_BIND_SERVICE \ --read-only \ --tmpfs /tmp:rw,noexec,nosuid,size=64m \ --memory=512m \ --cpus=1.0 \ --pids-limit=100 \ -p 8080:8080 \ myapp:latest ```
```

```
## 3. Pažeidžiamųjų skenavimas su Trivy ```bash trivy image --severity HIGH,CRITICAL myapp:latest trivy config Dockerfile ```
```

Botų ir Webhook Saugumo Standartai:

```
## 1. Telegram Webhook Secret Token tikrinimas
```

```
```python from fastapi import FastAPI, Request, HTTPException SECRET_TOKEN = "LabaiSlaptasRaktas123456789" @app.post("/webhook") async def telegram_webhook(request: Request): token =
```

```
request.headers.get("X-Telegram-Bot-Api-Secret-Token")
if token != SECRET_TOKEN: raise
HTTPException(status_code=403, detail="Netikras
siuntėjas!") return await process_update(await
request.json()) ````
```

```
2. Ugniasienės taisyklės Telegram potinkliams ```` bash
sudo ufw allow proto tcp from 149.154.160.0/20 to any
port 443 sudo ufw allow proto tcp from 91.108.4.0/22 to
any port 443 ````
```

```
3. Administratoriaus ID filtravimas ```` python
ADMIN_IDS = [123456789, 987654321]
@router.message(Command("admin")) async def
admin_handler(message: Message): if
message.from_user.id not in ADMIN_IDS: await
message.answer("Prieiga uždrausta.") return await
do_admin_work() ````
```

# Tinklo Žvalgybos ir Nmap Pažeidžiamumų Skenavimas:

```
1. Profesionalios Nmap komandos ```` bash
Slaptas SYN skenavimas: nmap -sS -p- -T4 server_ip
Servisų ir versijų atpažinimas: nmap -sV -sC -p
22,80,443,8080 server_ip
Pažeidžiamumų paieška su NSE skriptais: nmap --script
vuln server_ip
SSL/TLS šifrų patikra: nmap --script ssl-enum-ciphers -p
443 domain.lt ````
```

## 2. Pašto apsaugos (Anti-Spoofing) patikra `` ` bash dig +short TXT domain.lt | grep "v=spf1" dig +short TXT \_dmarc.domain.lt dig +short MX domain.lt `` `

## 3. Tinklo paketų fiksavimas su tcpdump `` ` bash sudo tcpdump -nn -i eth0 'tcp port 80 or tcp port 443' -c 50 sudo tcpdump -i eth0 -s 0 -w capture.pcap 'port 22 or port 443' `` `

# Python DevSecOps ir Priklausomybių Auditas --- ## 1. Priklausomybių saugumo skenavimas (pip-audit) `` ` bash pip install pip-audit pip-audit -r requirements.txt --desc on pip-audit --fail-on critical `` `

## 2. Statinė kodo analizė su Bandit (SAST) `` ` bash pip install bandit bandit -r . -ll -ii bandit -r . -f html -o bandit\_report.html `` `

## 3. Nesaugių funkcijų keitimas - ` pickle.loads() ` -> ` json.loads() ` - ` eval() ` / ` exec() ` -> ` ast.literal\_eval() ` - ` subprocess(shell=True) ` -> ` subprocess.run(['cmd', 'arg'], shell=False) ` - ` random ` -> ` secrets ` modulį (kriptografiniams raktams).

# Atsarginių Kopijų ir Ransomware Apsauga:

## 1. 3-2-1 Taisyklė - 3 kopijos - 2 skirtingos laikmenos - 1 nutolusi (off-site) ir NEPAKEIČIAMA (Immutable / WORM)

## 2. Šifruotas archyvavimas su age `` ` bash # Sukurti ir užšifruoti archyvą: tar -czf - /var/www /etc/nginx | age -r "age1ql3z7hgy..." > backup\_\$(date +%F).tar.gz.age #

Iššifruoti ir atkurti: age -d -i master\_key.txt backup\_2026-09-22.tar.gz.age | tar -xzf - -C /restore/ ```

## 3. Immutable bitas apsaugai nuo ištrynimo ``` bash  
sudo chattr +i backup.tar.gz sudo chattr -i backup.tar.gz  
```

Kriptografija ir Slaptų Duomenų Valdymas Terminale:

1. Failų šifravimas su age ``` bash age-keygen -o
secret_key.txt age -r age1ql3z7hjy... -o
slaptas_failas.txt.age slaptas_failas.txt age -d -i
secret_key.txt slaptas_failas.txt.age > atkurtas_failas.txt
```

## 2. Saugus failų sunaikinimas (Secure Wipe) ``` bash #  
Linux: shred -u -z -n 5 confidential\_data.env # macOS:  
srm -vz confidential\_data.env ```

## 3. Laikinas RAM diskas paslaptims ``` bash sudo  
mkdir -p /mnt/secure\_ram sudo mount -t tmpfs -o  
size=100M tmpfs /mnt/secure\_ram sudo umount  
/mnt/secure\_ram ```